Task 03 — API Error Handling

Implement standard error responses in your **Spring Boot microservice** using @RestControllerAdvice.

1. Create ErrorResponse.java

Create package:

src/main/java/com/example/<your-service>/exception

Create **ErrorResponse.java**:

```
package com.example.orderservice.exception;

import java.time.LocalDateTime;

public class ErrorResponse {

  private LocalDateTime timestamp;
  private int status;
  private String message;
  private String path;

  public ErrorResponse() {
  }

  public ErrorResponse(LocalDateTime timestamp, int status,
              String message, String path) {
    this.timestamp = timestamp;
    this.status = status;
    this.message = message;
    this.path = path;
  }

  public LocalDateTime getTimestamp() {
    return timestamp;
  }
```

```
    public void setTimestamp(LocalDateTime timestamp) {
        this.timestamp = timestamp;
    }

    public int getStatus() {
        return status;
    }

    public void setStatus(int status) {
        this.status = status;
    }

    public String getMessage() {
        return message;
    }

    public void setMessage(String message) {
        this.message = message;
    }

    public String getPath() {
        return path;
    }

    public void setPath(String path) {
        this.path = path;
    }
}
```

Replace com.example.orderservice with your actual package name.

2. Create GlobalExceptionHandler.java

In the same exception package:

```
package com.example.orderservice.exception;

import jakarta.servlet.http.HttpServletRequest;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.LocalDateTime;

@RestControllerAdvice
public class GlobalExceptionHandler {

  private ResponseEntity<ErrorResponse> buildErrorResponse(
      HttpStatus status,
      String message,
      HttpServletRequest request) {

    ErrorResponse error = new ErrorResponse(
        LocalDateTime.now(),
        status.value(),
        message,
        request.getRequestURI()
    );

    return new ResponseEntity<>(error, status);
  }

  // 400 - Bad Request
  @ExceptionHandler(IllegalArgumentException.class)
  public ResponseEntity<ErrorResponse> handleBadRequest(
      IllegalArgumentException ex,
      HttpServletRequest request) {

    return buildErrorResponse(
        HttpStatus.BAD_REQUEST,
        ex.getMessage(),
        request
```

```
    );
}

// 401 - Unauthorized
@ExceptionHandler(SecurityException.class)
public ResponseEntity<ErrorResponse> handleUnauthorized(
      SecurityException ex,
      HttpServletRequest request) {

    return buildErrorResponse(
          HttpStatus.UNAUTHORIZED,
          ex.getMessage(),
          request
    );
}

// 403 - Forbidden
@ExceptionHandler(AccessDeniedException.class)
public ResponseEntity<ErrorResponse> handleForbidden(
      AccessDeniedException ex,
      HttpServletRequest request) {

    return buildErrorResponse(
          HttpStatus.FORBIDDEN,
          ex.getMessage(),
          request
    );
}

// 404 - Not Found
@ExceptionHandler(ResourceNotFoundException.class)
public ResponseEntity<ErrorResponse> handleNotFound(
      ResourceNotFoundException ex,
      HttpServletRequest request) {

    return buildErrorResponse(
          HttpStatus.NOT_FOUND,
          ex.getMessage(),
```

```
            request
        );
    }

    // 409 - Conflict
    @ExceptionHandler(ConflictException.class)
    public ResponseEntity<ErrorResponse> handleConflict(
            ConflictException ex,
            HttpServletRequest request) {

        return buildErrorResponse(
                HttpStatus.CONFLICT,
                ex.getMessage(),
                request
        );
    }
}
```

3. Create the custom exceptions

ResourceNotFoundException.java

```
package com.example.orderservice.exception;


public class ResourceNotFoundException extends RuntimeException {


    public ResourceNotFoundException(String message) {
        super(message);
    }
}
```

ConflictException.java

```
package com.example.orderservice.exception;
```

```
public class ConflictException extends RuntimeException {

   public ConflictException(String message) {
      super(message);
   }
}
```

AccessDeniedException.java

```
package com.example.orderservice.exception;

public class AccessDeniedException extends RuntimeException {

   public AccessDeniedException(String message) {
      super(message);
   }
}
```

4. Use exceptions in your service/controller

For example:

```
if (order == null) {

   throw new ResourceNotFoundException("Order not found");

}
```

For a duplicate order:

```
if (orderAlreadyExists) {

   throw new ConflictException("Order already exists");

}
```

For invalid input:

```
if (orderId <= 0) {
   throw new IllegalArgumentException("Invalid order ID");
}
```

5. Expected API response

For a **404** request such as:

```
GET /orders/999
```

the response will be:

```
{
  "timestamp": "2026-09-07T14:38:00",
  "status": 404,
  "message": "Order not found",
  "path": "/orders/999"
}
```

Status codes to handle

Status	Meaning	Example
400	Bad Request	Invalid input
401	Unauthorized	Authentication required
403	Forbidden	User has no permission
404	Not Found	Order/user doesn't exist
409	Conflict	Duplicate resource

Task 3 is complete when these standard error responses are returned consistently from your API.